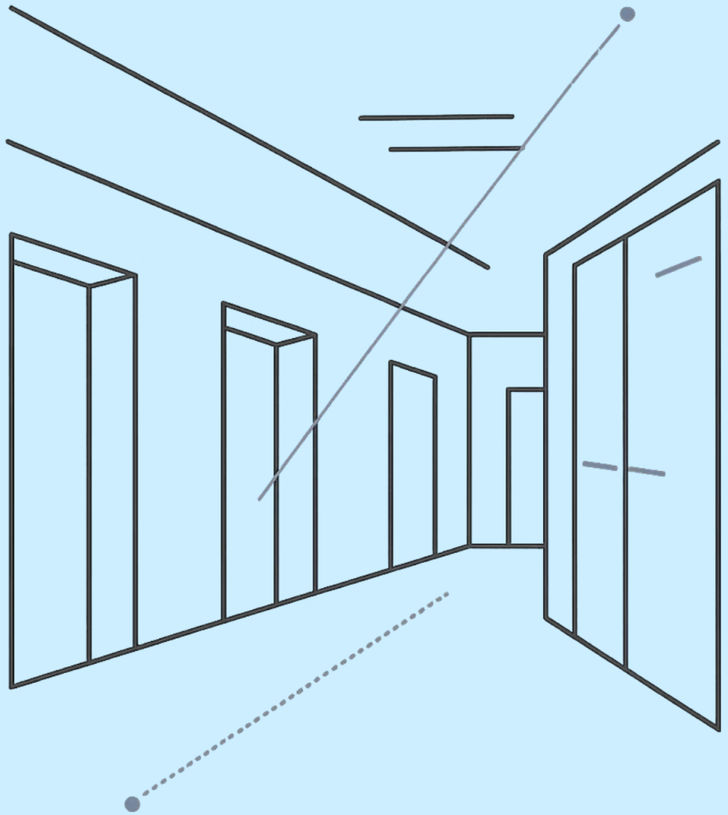


Hostel Complaint Management System

Maintenance Process Model

A structured approach to managing, tracking, and resolving hostel maintenance complaints through a systematic and iterative software development process.

Team Member 1

Nikhilesh Reddy

(RA2311027010015)

Team Member 2

Dhwani Agarwal

(RA2311027010021)

Team Member 3

Rajveer Singh

(RA2311027010038)

Selected Maintenance Process Model

Iterative Enhancement Model

This model focuses on improving the system through repeated iterations and updates. Rather than building the entire system at once, development is broken down into manageable cycles — each one refining and expanding upon the last.

The Iterative Enhancement Model is widely recognized in software engineering as one of the most practical approaches for systems that must remain responsive to evolving user needs. Each iteration delivers a working version of the software that can be tested, evaluated, and improved before the next phase begins.

For a dynamic environment such as a hostel complaint management system — where user needs, maintenance workflows, and administrative processes are subject to change — this model provides the ideal balance between structured planning and adaptive flexibility.

Core Principles

→ **Plan**

Define scope and objectives for each iteration cycle

→ **Develop**

Implement features incrementally with measurable goals

→ **Evaluate**

Test, review feedback, and identify improvements

→ **Enhance**

Refine the system and begin the next iteration

JUSTIFICATION

Why the Iterative Enhancement Model?

The Iterative Enhancement Model was selected for the Hostel Complaint Management System based on a careful analysis of the project's nature, scale, and long-term requirements. Complaint management systems are inherently dynamic — user expectations shift, administrative workflows evolve, and new features are regularly requested. A rigid, single-pass development model cannot adequately accommodate this reality.

The following key factors make this model the ideal choice for our project:

Evolving System Requirements

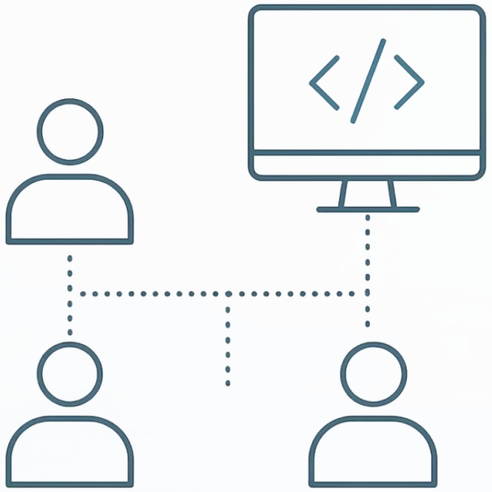
The system is designed to grow over time. New modules, features, and integrations can be introduced in subsequent iterations without disrupting the existing codebase.

Continuous Improvement

Each iteration provides an opportunity to review performance, collect user feedback, and implement targeted enhancements that improve overall quality.

Bug Fixing & Ongoing Updates

Issues identified during testing can be addressed within the same iterative cycle, ensuring the system remains stable and reliable throughout development.



Adapts to Changing User Requirements

Hostel administrators and residents may request changes to workflows or reporting features. The iterative model accommodates these mid-course corrections gracefully.

Ideal for Dynamic Applications

Complaint management systems require real-time responsiveness and ongoing updates. The iterative model is purpose-built for applications that must remain live and relevant throughout their lifecycle.

The Iterative Enhancement Model ensures that every version of the system delivered to users is functional, tested, and incrementally better than the last.

APPLICATION

Applying the Iterative Enhancement Model

The development of the Hostel Complaint Management System is organized into four distinct iterations, each building upon the previous to deliver a progressively more capable and refined system. This step-by-step evolution ensures that core functionality is established early, while advanced features are introduced in a controlled and well-tested manner.

Iteration 1

Basic Login & Complaint Submission

Establish the foundational layer of the system. Implement user authentication (student and admin login), basic complaint submission forms, and backend database structure. Users can register, log in, and submit complaints with a description and category.

1

Iteration 2

Complaint Tracking & Status Updates

Introduce complaint lifecycle management. Users can view the current status of their submitted complaints (Pending, In Progress, Resolved). Admin staff can update complaint statuses, and basic filtering by category or date is enabled.

2

Iteration 3

Admin Dashboard & Assignment System

Build a comprehensive admin control panel. Administrators gain the ability to assign complaints to specific maintenance personnel, set priority levels, view all active and resolved complaints, and generate basic reports on system activity.

3

Iteration 4

Notifications & Final Improvements

Implement automated email or in-app notifications to alert users when their complaint status changes. Conduct full system testing, address bugs discovered in previous iterations, and refine the user interface based on stakeholder feedback.

4

Each iteration produces a fully functional, testable version of the system — ensuring quality is maintained at every stage of development.

Feedback collected at the end of each iteration directly informs the scope and priorities of the next cycle.

Advantages of the Iterative Enhancement Model

The Iterative Enhancement Model offers a compelling set of advantages that make it exceptionally well-suited for the development and long-term maintenance of the Hostel Complaint Management System. These benefits extend beyond the development phase and support the ongoing health and usability of the software throughout its operational life.

Easy Maintenance & Updates

Because the system is built in modular, well-defined increments, individual components can be updated or replaced without disrupting the entire application. Routine maintenance tasks — such as patching bugs, updating logic, or upgrading dependencies — are straightforward and low-risk.

Flexible Development Process

The iterative approach allows the development team to respond to changing requirements without starting over. Scope adjustments, feature reprioritization, and design revisions can all be incorporated into future iterations with minimal overhead and disruption.

Supports Continuous Improvement

Each completed iteration provides an opportunity to reassess quality, collect user feedback, and introduce refinements. This creates a built-in culture of improvement where the system consistently becomes more capable, more reliable, and more aligned with user expectations over time.

Reduces Development Risks

By delivering working software in smaller increments, the team can identify and address issues early — before they compound into larger, more costly problems. This risk-reduction approach is especially valuable in student projects where timelines and resources are constrained.

Improves User Satisfaction

Regular delivery of new features and visible progress keeps stakeholders engaged and confident in the system's trajectory. Users see tangible improvements with each iteration, fostering trust and encouraging constructive feedback that further drives quality enhancements.